CS 3006
Parallel and Distributed Computing
Lecture 12 + 13

Danyal Farhat

FAST School of Computing

NUCES Lahore

Synchronization Constructs in OpenMP

Loop Worksharing Construct

- Sequential Code
  - for(i=istart;I<iend;i++) { a[i] = a[i] + b[i];}
- OpenMP Parallel Region
  - #pragma omp parallel
    - {
    - int id, i, Nthrds, istart, iend;
    - id = omp_get_thread_num();
    - Nthrds = omp_get_num_threads();
    - istart = id * N / Nthrds;
    - iend = (id+1) * N / Nthrds;
    - for(i=start;i<iend; i++) {a[i] = a[i] + b[i];}
    - }
- OpenMP parallel region and a worksharing for construct
  - #pragma omp parallel
    - #pragma omp for
    - for(i=0;I<N;i++) {
    - a[i] = a[i] + b[i];}

Numerical Integration: Pi program

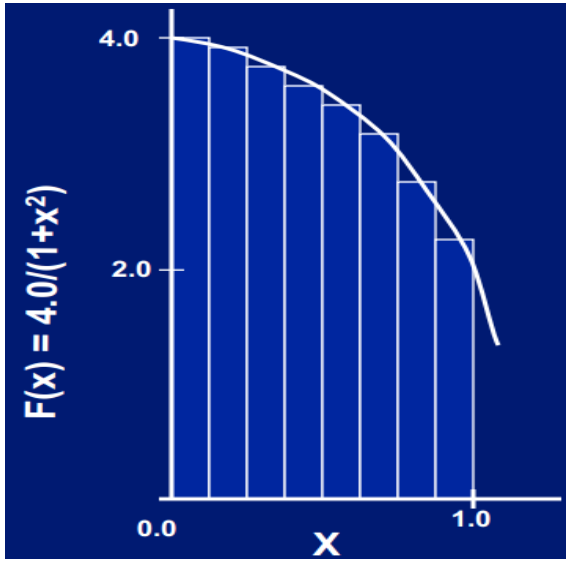
- **Mathematically**

$$\int_0^1 \frac{4.0}{(1+x^2)}\,dx = \pi$$

- **Integral can be approximated as a sum of rectangles:**

$$\sum_{i=0}^{N} F(x_i)\Delta x \approx \pi$$

- **Where each rectangle has**
  - **width Δx and height F(xi) at**
  - **the middle of interval i.**



Source: http://openmp.org/mp-documents/omp-hands-on-SC08.pdf

Serial Pi Program

```
static long num_steps = 100000; // total rectangles
double step;
void main ()
{
  int i; double x, pi, sum = 0.0;
  step = 1.0/(double) num_steps; // size of one rectangle
  for (i=0;i< num_steps; i++){
    x = (i+0.5)*step; // rectangle width
    sum = sum + 4.0/(1.0+x*x); // rectangle area // integrating area
 }
    pi = step * sum;
}
```

Parallel Pi Program

```
static long num_steps = 100000;
double step;
#define NUM_THREADS 4
int _tmain(int argc, _TCHAR* argv[])
{
  int i, nthreads;
  double pi, sum[NUM_THREADS];
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS); // Setting number of threads
```

Parallel Pi Program (Cont.)

```
#pragma omp parallel
 {
   int i, id, nthrds;
   double x;
   id = omp_get_thread_num();              // Getting thread Id (Unique Id for each thread)
   nthrds = omp_get_num_threads();         // Number of threads in the program
   if(id == 0)
     nthreads = nthrds;
   for (i=id, sum[id]=0.0;i< num_steps; i=i+nthrds){
       x = (i+0.5)*step;  // rectangle width
       sum[id] += 4.0/(1.0+x*x); // array with partial sum of each thread
     }
 }
```

Parallel Pi Program (Cont.)

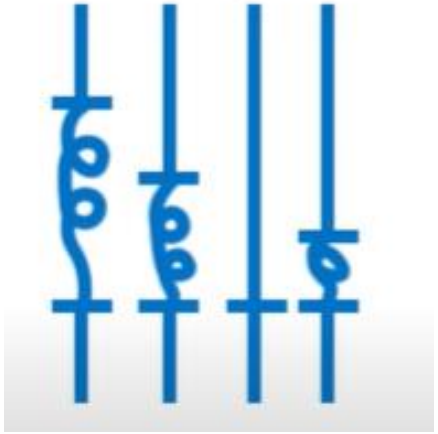
```
for(i=0,pi=0.0; i<nthreads; i++)
        pi += sum[i] * step;
    printf("%f\n", pi);
    return 0;
}
```

Synchronization Constructs

- **Barrier**
- **Mutex**
- **Atomic**

Barrier Construct

- **Barrier:** **Each thread wait at the barrier until all threads arrive**



```
#pragma omp parallel

{
        int id=omp_get_thread_num();
        A[id] = big_cal1(id);
#pragma omp barrier

        B[id] = big_calc2(id, A);
}
```

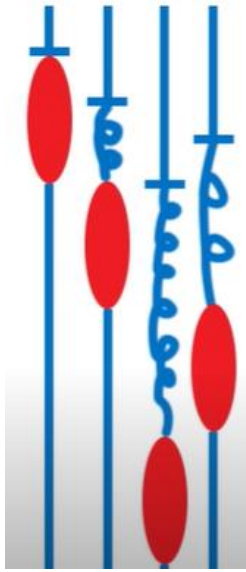
Mutex Construct

- **Mutual Exclusion:** **Only one thread at a time can enter critical section**

```
#pragma omp critical [name]
  {
    code_block
  }
```

- **[name] optional name that identifies the critical directive**

Mutex Construct (Cont.)



Mutual Exclusion:
Define a block of code that only one thread at a time can execute

```
float res;

#pragma omp parallel

[       float B; int i, id, nthrds;
        id = omp_get_thread_num();
        nthrds = omp_get_num_threads();
        for(i=id; i<niters; i+=nthrds){
            B = big_job(i);

#pragma omp critical
            res += consume (B);
        }
}
```

Parallel Pi Program using Mutual Exclusion

```
static long num_steps = 100000;
double step;
#define NUM_THREADS 4
int _tmain(int argc, _TCHAR* argv[])
{
  int i, nthreads;
  double pi, sum[NUM_THREADS];
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS); // Setting number of threads
```

Parallel Pi Program using Mutual Exclusion (Cont.)

```
#pragma omp parallel
{
  int i, id, nthrds;
  double x;
  id = omp_get_thread_num();          // Getting thread Id (Unique Id for each thread)
  nthrds = omp_get_num_threads();     // Number of threads in the program
  if(id == 0)
    nthreads = nthrds;
  for (i=id, sum[id]=0.0;i< num_steps; i=i+nthrds){
     x = (i+0.5)*step;  // rectangle width
     sum[id] += 4.0/(1.0+x*x); // array with partial sum of each thread
    }
}
```

Parallel Pi Program using Mutual Exclusion (Cont.)

```
#pragma  omp critical                    // Ensuring Mutual Exclusion
    pi += (sum * step);
 }
      printf("%f\n", pi);
  }
```

Atomic Construct

```
#pragma omp parallel for
 {
        for (i=0; i++, i<n)
        {
                #pragma omp atomic          // Shortcut for critical section
                ic = ic + bigfunc();        // Little faster than critical overhead
        }
        printf("counter = %d\n", ic);
 }
```

Supported functions

+, *, -, /, &, ^, |, <<, >>

Reduction

- Sometimes there is true dependency between loop iterations which can not be removed.
- double sum=0.0; ave=0.0; A[MAX]; int i;

```
for (i=0;i< MAX; i++) {
sum + = A[i];
}
ave = sum/MAX;
```

- We are combining values into a single accumulation variable (sum)
- How to share work among threads. Such situation is called Reduction.

OpenMP Reduction Clause

- **reduction (op : list)**
- **Inside a parallel work a local copy of each list variable is made**
- **List variable is initialized depending on the “op”**

  (e.g. 0 for “+”)
- **Compiler updates each local copy**
- **Local copies are reduced into a single value and combined with the original global value**

```
double ave=0.0; sum=0.0; A[MAX]; int i;
#pragma omp parallel for reduction (+:sum)
for (i=0;i< MAX; i++) {sum + = A[i];}
ave = sum/MAX;
```

Pi Program using Reduction Clause

```
static long num_steps = 100000;
double step;
void main () {
  int i; double pi, sum = 0.0;
  step = 1.0/(double) num_steps;
#pragma omp parallel {
double x;
#pragma omp for reduction(+:sum)          // reduction (op : list) is equivalent to reduction (+:sum)
  for (i=0;i< num_steps; i++){
    x = (i+0.5)*step;
    sum = sum + 4.0/(1.0+x*x);
  } }
#pragma  omp critical                     // Ensuring Mutual Exclusion
    pi = step * sum;
}
```

Sections Constructs

```
#pragma omp parallel
{
   #pragma omp sections
   {
         #pragma omp section
         (void) funcA();  // x-calculation

         #pragma omp section
         (void) funcB(); // y-calculation
   } /*-- End of sections block --*/
} /*-- End of parallel region --*/
```

Single Constructs

```
#pragma omp parallel shared(a,b) private(i)
{
        #pragma omp single    // Only one thread can ignore barrier while others stop at barrier
{
                a = 10;
                printf("Single construct executed by thread %d\n",
        omp_get_thread_num());
         }

/* A barrier is automatically inserted here */

        #pragma omp for
        for (i=0; i<n; i++)
                b[i] = a;

} /*-- End of parallel region --*/
```

Barrier and NoWait Constructs

Barrier

```
#pragma omp barrier
```

Nowait

```
#pragma omp for nowait
for(i=0; i<n; i++)
 {}
// next task
```

Combined Constructs

Full version	Combined construct

#pragma omp parallel
{
#pragma omp for
for-loop
}
	#pragma omp parallel for
	for-loop

#pragma omp parallel
{
#pragma omp sections
{
[#pragma omp section]
structured block
[#pragma omp section
structured block]
...
}
}
	#pragma omp parallel sections
	{
	[#pragma omp section]
	structured block
	[#pragma omp section
	structured block]
	...
	}

Synchronization Locks

- **omp init lock()**
- **omp set lock()**
- **omp_unset_lock()**
- **omp_test_lock()**
- **omp_destroy_lock()**

OpenMP Variable Scope

Private:

- **A private copy will be created for each thread without assigning any value.**

Firstprivate:

- **A private copy will be created for each thread initialized with already assigned value (if any).**
  - **Master thread's value**

OpenMP Variable Scope (Cont.)

Lastprivate:

- **Value produced in the last iteration by one of the multiple threads will be assigned to the variable which will be accessible outside the parallel region**

Shared:

- **It is the default scope and such a variable will be shared among all the threads**

The Schedule Clause

Static:

- **Pre-determined and predictable by the programmer**
- **Scheduling done at compile time**
- **Least work at runtime**

- **#pragma omp for schedule(static, chunk_size)**
- **#pragma omp for schedule(static)**

The Schedule Clause (Cont.)

Dynamic:

- **Unpredictable and mostly used when task size is unknown**
- **Complex scheduling used at run-time**
- **Most work at runtime**

- **#pragma omp for schedule(dynamic, chunk_size)**
- **#pragma omp for schedule(dynamic)**

The Schedule Clause (Cont.)

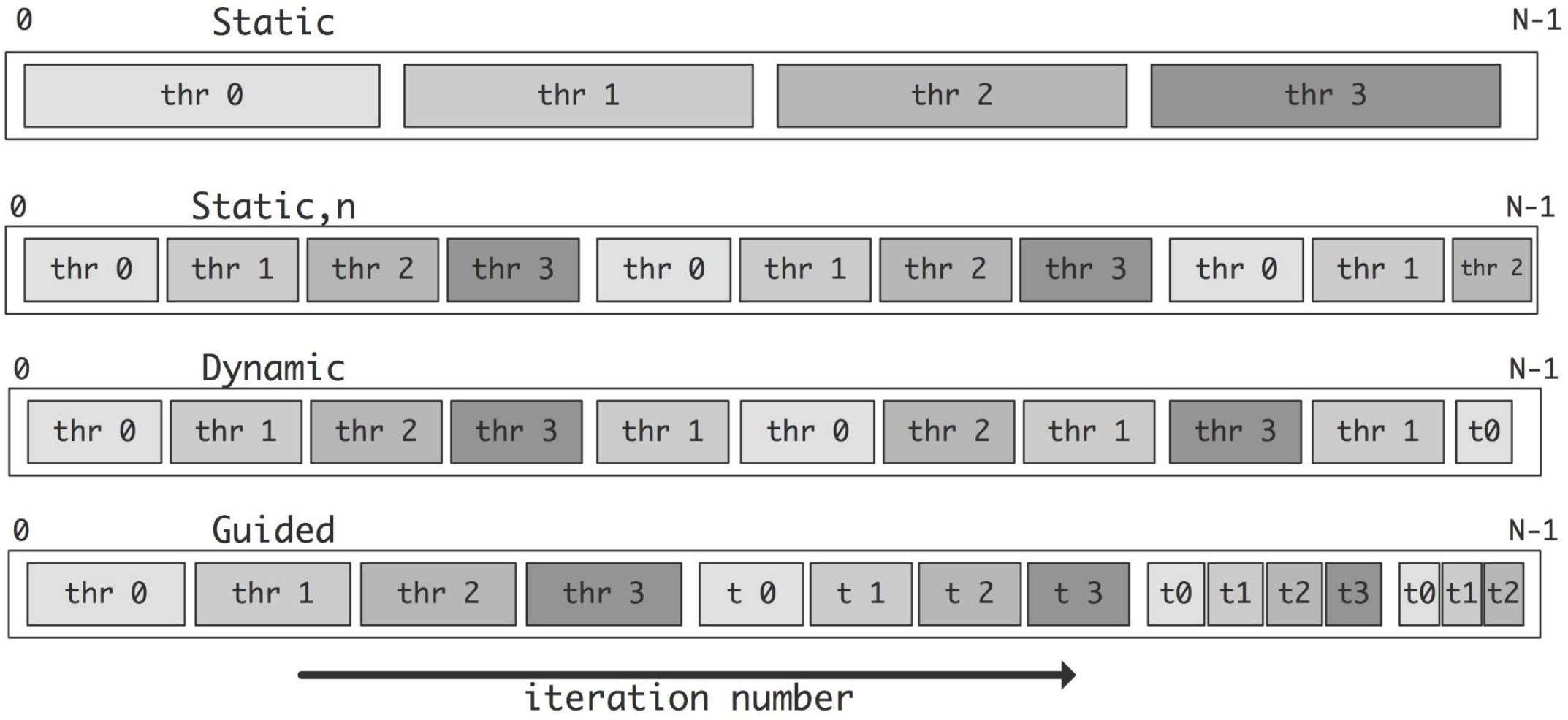
Guided:

- **Special case of dynamic to reduce scheduling overhead**
- **Larger chunks are selected at start and then its size is reduced to get better distribution**

Runtime:

- **Scheduling technique could be selected at runtime by using this clause**

The Schedule Clause (Cont.)



Runtime Environment Routines

- **Modify/Check the number of threads**

  omp_set_num_threads(), omp_get_num_threads(),
  omp_get_thread_num(), omp_get_max_threads()

- **Are we in an active parallel region?**
  - **omp_in_parallel()**
- **How many processors in the system?**
  - **omp_num_procs()**
- **Nested Parallelism**

  Om_set_nested(1)

  Omp_set_max_active_levels()

Additional Resources

- **Book: Introduction to Parallel Computing by Ananth Grama and Anshul Gupta**

- **Chapter 7: Programming Shared Address Space Platforms**

- **Introduction to OpenMP by Tim Mattson (Intel)**
  - **https://www.youtube.com/watch?v=nE-xN4Bf8XI&list=PLLX-Q6B8xqZ8n8bwjGdzBJ25X2utwnoEG&index=1**

Questions?

Class Activity